Semantic Token Space (STS): A Multidimensional Formalization of Semantic Intermediate Representations

Abstract

Traditional tokenization methods such as subword segmentation are effective for sequence modeling, but they are weak as explicit representations of structure and meaning. They preserve lexical decomposability and statistical efficiency while leaving syntax, semantics, and logical dependency largely implicit. This paper develops a deterministic alternative based on compilation rather than segmentation. We present the Semantic Tokenization Framework (STF) and its Semantic Intermediate Representation (SIR), in which a document is compiled into semantic tokens, typed relations, and validation metadata. We then extend the operational model into the Semantic Token Space (STS), a multidimensional formalization in which tokens are points in a coupled space of lexical, syntactic, semantic, logical, contextual, pragmatic, temporal, and coherence dimensions. The main contributions are fivefold: a deterministic semantic intermediate representation; a formal validation model based on schema conformance and invariants; a graph projection, SirGraph, that materializes the relation-bearing view of compiled artifacts without inference; a retention metric ρ for dimension-wise preservation analysis; and a mathematical formalization of semantic tokens as constrained elements of a valid manifold. The resulting framework connects representation theory, compiler construction, and AI system design, and provides a stable baseline for structured retrieval, agent reasoning, semantic differencing, and future semantic query engines.

1 Introduction

Most contemporary language-processing systems consume text as flat token streams. This design is computationally convenient, but it places a structural burden on downstream components. Syntax must be reconstructed from sequence position, semantic units must be inferred from distributed state, and logical relations must be recovered repeatedly from untyped surface material. As a result, retrieval systems operate on coarse chunks, reasoning systems repeatedly parse the same evidence, and agent systems often lack a stable intermediate form over which they can validate, compare, or compose semantic state.

This problem is not a failure of subword tokenization relative to its original purpose. Methods such as byte-pair encoding and WordPiece were designed to balance vocabulary coverage and sequence efficiency. They were not designed to serve as explicit, deterministic representations of syntax, semantics, and logical structure. When they are treated as such, a mismatch appears between sequence encoding and meaning-preserving representation.

STF addresses this mismatch by treating document processing as compilation into an intermediate representation. The compiled object is the SIR, a deterministic artifact composed of semantic tokens, typed relations, and reproducibility metadata. The core compiled unit is the *ztoken*, written

$$z = \langle L, S, \Sigma, \Phi \rangle,$$

where L is lexical information, S is syntactic structure, Σ is semantic content, and Φ is the logical dimension in the v1 formulation. Throughout the present paper, Γ is used as the preferred symbol for the generalized logical dimension, with Φ retained as a legacy alias on the operational slice.

The paper makes the following contributions.

- It formalizes STF-SIR as a deterministic compilation model for semantic intermediate representations.
- It defines an artifact model with validation, invariants, and stable serialization.

- It characterizes SirGraph as a graph projection over compiled artifacts rather than an inferred knowledge graph.
- It introduces a retention metric ρ for dimension-wise preservation analysis.
- It defines the Semantic Token Space (STS) as a multidimensional semantic space together with a valid token manifold, coupling functions, and algebraic operations.

Taken together, these components yield a representation framework in which semantic units can be addressed, validated, compared, and transformed without collapsing immediately back to raw text. The aim is not to replace statistical language models, but to provide a rigorous intermediate layer between source documents and intelligent computation. We position STF-SIR as a deterministic semantic intermediate representation for document-centric AI systems.

2 STF-SIR Operational Model

ZTokens

The v1 operational unit is the ztoken

$$z = \langle L, S, \Sigma, \Phi \rangle.$$

The four coordinates are interpreted as follows:

- L: lexical realization, including source span, surface text, and normalized text;
- S: syntactic placement, including node type, depth, parent reference, and structural path;
- Σ: semantic content, represented in v1 by a deterministic gloss-centered fallback;
- Φ: logical references local to the token, together with global relations in the artifact.

In the generalized theory, Φ is identified with the logical dimension Γ. Thus,

$$\Phi_{\mathrm{v1}} \equiv \Gamma_{\mathrm{STS}}.$$

A ztoken is the primary compiled unit. It is not definitionally identical to an arbitrary graph node. Rather, it is a stable compiled carrier that can be materialized as a graph node in SirGraph.

Compilation Pipeline

Let $\mathfrak{L}$ denote the space of admissible source documents. To avoid ambiguity, $\mathfrak{L}$ denotes the document language space, while $\mathcal{L}$ denotes the lexical coordinate space used later in STS. The compiler is written

$$C_\theta : \mathfrak{L} \rightharpoonup \mathcal{Z},$$

where θ fixes parser behavior, normalization rules, and serialization policy.

Operationally, the pipeline is

$$C_\theta = \mathsf{Log} \circ \mathsf{Sem} \circ \mathsf{Syn} \circ \mathsf{Lex},$$

where:

- Lex extracts deterministic lexical spans and normalized text;
- Syn constructs the document structure from a CommonMark-compatible parse;
- Sem assigns the v1 semantic fallback;
- Log emits typed relations.

The current implementation supports a minimal set of block-level constructs, including headings, paragraphs, block quotes, lists, list items, and code blocks. For a token with lexical plain text $L.\text{plain_text}$, the v1 semantic fallback is

$$\Sigma.\text{gloss} = \begin{cases} \text{normalize}(L.\text{plain_text}) & \text{if } L.\text{plain_text} \neq \varepsilon, \\ \varepsilon & \text{otherwise,} \end{cases}$$

where normalize denotes deterministic Unicode normalization, trimming, and whitespace collapse, and ε denotes the empty string.

Determinism Guarantees

The operational baseline is deterministic under fixed input, implementation version, and compilation parameters. In particular:

- ztoken identifiers are stable and emitted in traversal order as $z_1, z_2, \ldots$;
- relation identifiers are stable and emitted in traversal order as $r_1, r_2, \ldots$;
- traversal order is fixed by deterministic preorder over the parsed structure;
- serialization is stable, UTF-8 encoded, and field-ordered;
- timestamps, random identifiers, and nondeterministic iteration are excluded.

Accordingly, if $d \in \mathfrak{L}$ and two runs share the same θ and implementation state, then their serialized outputs are byte-identical:

$$\text{ser}(C_\theta^{(i)}(d)) = \text{ser}(C_\theta^{(j)}(d)).$$

Validation Model

Validation is factored into schema conformance and invariants. The schema checks artifact shape and required fields. The invariants check semantic and structural well-formedness, including:

- unique token and relation identifiers;
- valid byte and line spans;
- count consistency between declared counts and materialized objects;
- valid references for parent identifiers and relation endpoints;
- compliance with the semantic fallback rule.

These validation results later contribute to the coherence coordinate Ω.

3 Artifact Model

For a compiled document d, the operational artifact is

$$\mathcal{A}_d = (Z_d, R_d, M_d),$$

where:

- $Z_d = (z_1, \ldots, z_n)$ is an ordered sequence of ztokens;
- $R_d = (r_1, \ldots, r_m)$ is an ordered sequence of global relations;
- M_d is metadata sufficient for provenance and reproducibility.

The canonical serialized form is the `.zmd` document, a UTF-8 YAML artifact with deterministic field ordering. It is the interchange format for compiled SIR artifacts, while the in-memory `Artifact` object is the canonical typed representation in the implementation.

Each relation has the form

$$r = (\mathrm{id}, \mathrm{type}, \mathrm{category}, \mathrm{source}, \mathrm{target}, \mathrm{stage}).$$

The category field is typed in

$$\{\texttt{structural}, \texttt{logical}, \texttt{semantic-link}\}.$$

In the current baseline, the emitted relation types are `contains` and `precedes`, both classified as `structural`. The stage field records the pipeline stage that emitted the relation; it is a provenance field rather than a semantic type. Thus a relation may be `category: structural` and `stage: logical` when it is emitted during the logical stage.

The artifact model deliberately separates local token structure from global relations. A ztoken carries its own four-dimensional local content, while the artifact records document-level order, metadata, and cross-token connectivity. This separation makes serialization stable and query surfaces predictable.

Illustrative Example

The operational flow from source document to serialized artifact and graph projection can be illustrated with a minimal example. Consider the Markdown input:

```
# Semantic IR

Deterministic structure matters.
```

A corresponding `.zmd` fragment may be sketched as:

```
ztokens:
  - id: z1
    syntactic:
      node_type: heading
    semantic:
      gloss: "Semantic IR"
  - id: z2
    syntactic:
      node_type: paragraph
    semantic:
      gloss: "Deterministic structure matters."
relations:
  - id: r1
    type: precedes
    category: structural
    source: z1
    target: z2
    stage: logical
```

The corresponding graph projection is then

$$\mathsf{SirGraph}(\mathcal{A}_d) = (V_d, E_d), \quad V_d = \{z_1, z_2\}, \quad E_d = \{(z_1 \xrightarrow{\texttt{precedes}} z_2)\}.$$

This example is intentionally minimal. Its purpose is to show that the same compiled artifact supports both stable serialization and deterministic graph realization.

4 SirGraph

SirGraph is the first graph-oriented realization of STF-SIR. It is computed from an artifact rather than independently compiled or serialized. Let

$$\mathcal{A}_d = (Z_d, R_d, M_d).$$

If $Z_d = (z_1, \ldots, z_n)$, then each compiled token satisfies $z_i \in \mathcal{M}_{\mathrm{STF}}$ at the level of the token formalization introduced later. The graph realization is defined directly from the artifact by

$$\mathsf{SirGraph}(\mathcal{A}_d) = \Pi_{\Sigma,\Gamma}(\mathcal{A}_d) = (V_d, E_d),$$

with:

$$V_d = \{v_i \mid z_i \in Z_d\}, \qquad E_d = \{e_j \mid r_j \in R_d\}.$$

Each token yields exactly one node, and each relation yields exactly one edge. Node and edge identifiers are preserved exactly from the artifact.

At the level of abstraction, the graph view may also be summarized as

$$\mathsf{SirGraph} \approx \Pi_{\Sigma,\Gamma}(\mathcal{M}_{\mathrm{STF}}),$$

where the approximation symbol indicates that graph nodes preserve compiled-token identity rather than quotienting directly over abstract semantic objects. This distinction matters: a ztoken is a compiled unit and a potential graph anchor, but not every future graph node need be a ztoken.

The current graph layer satisfies three properties.

Determinism. Node order follows artifact token order; edge order follows artifact relation order; incoming and outgoing indexes are built deterministically.

No inference. SirGraph does not add auxiliary nodes, inferred edges, semantic closure, or external knowledge. It is a materialized view over the compiled artifact.

Projection only. The graph is not a second canonical representation. The canonical forms remain the typed artifact and its `.zmd` serialization; the graph is a derived view.

These constraints make SirGraph suitable as a stable graph baseline while preserving fidelity to the operational system.

5 Retention (ρ)

Retention is the metric layer of the framework. It evaluates how much of each representational dimension is preserved by the compiled artifact. At the token level, define

$$\rho(t) = \big(\rho_L(t), \rho_S(t), \rho_\Sigma(t), \rho_\Gamma(t)\big),$$

with each component in $[0, 1]$.

The intended interpretation is:

- ρ_L: lexical preservation, including valid spans and normalization consistency;
- ρ_S: syntactic preservation, including valid node type, path, and parent references;
- ρ_Σ: semantic preservation, currently measured by compliance with the deterministic gloss fallback;
- ρ_Γ: logical preservation, including relation completeness and reference consistency.

At the artifact level, the current implementation computes a deterministic baseline by aggregating completeness and consistency over all tokens and relations. The retention baseline is therefore operational rather than epistemically complete: it measures whether the artifact is internally faithful to the current semantics, not whether it matches an external gold-standard interpretation. More precisely, ρ measures internal representational fidelity, not external semantic correctness.

Two limitations should be made explicit. First, the present baseline does not compare against human semantic judgments or benchmark annotations. Second, the current ρ_Σ is constrained by the simplified semantic fallback and therefore should not be read as a full semantic adequacy measure. Nonetheless, ρ provides a useful metric layer that is stable, reproducible, and immediately connected to implementation invariants.

6 Semantic Token Space (STS)

The generalized semantic token is not restricted to the four-dimensional operational slice. Instead, define the Semantic Token Space as

$$\mathcal{T} = \mathcal{L} \times \mathcal{S} \times \mathcal{S}_{\text{em}} \times \mathcal{G} \times \mathcal{C} \times \mathcal{P} \times \mathcal{D} \times \mathcal{O}.$$

An element of $\mathcal{T}$ is written

$$t = (L, S, \Sigma, \Gamma, C, P, \Delta, \Omega).$$

The eight coordinates have the following interpretation:

- $L \in \mathcal{L}$: lexical evidence;
- $S \in \mathcal{S}$: syntactic structure;
- $\Sigma \in \mathcal{S}_{\text{em}}$: semantic meaning;
- $\Gamma \in \mathcal{G}$: logical relations;
- $C \in \mathcal{C}$: contextual dependency;
- $P \in \mathcal{P}$: pragmatic intent;
- $\Delta \in \mathcal{D}$: temporal or evolutionary state;
- $\Omega \in \mathcal{O}$: coherence and validation state.

The logical coordinate uses Γ as the preferred notation. The legacy symbol Φ is retained only when referring specifically to the operational v1 slice.

To reason about information ordering, assume that semantic meaning is partially ordered:

$$(\mathcal{S}_{\text{em}}, \preceq).$$

Here $\Sigma_1 \preceq \Sigma_2$ means that Σ_2 is at least as informative as Σ_1. We write $\succeq$ for the converse order. When needed, implementation-dependent information preorders may also be introduced on $\mathcal{G}$ and $\mathcal{O}$.

The coherence coordinate is modeled as

$$\Omega = (\omega_{\text{schema}}, \omega_{\text{inv}}, \omega_\rho) \in \mathcal{O},$$

with

$$\mathcal{O} = \{0,1\} \times \{0,1\} \times [0,1].$$

The three components represent schema validity, invariants validity, and retention-derived coherence, respectively. Since coherence is often artifact-relative, the more precise notation $\Omega_{\mathcal{A}}(t)$ may be used when the dependency on a compiled artifact matters. In the current implementation, the first two coordinates are determined by validation, while the third is induced by retention.

For coherence comparisons, equip $\mathcal{O}$ with the componentwise order:

$$(a_1, a_2, a_3) \leq (b_1, b_2, b_3) \quad \Longleftrightarrow \quad a_i \leq b_i \text{ for all } i \in \{1,2,3\}.$$

7 Valid Token Manifold

Not every point in the full product space $\mathcal{T}$ is a valid semantic token. Many tuples are mathematically formable yet semantically incoherent. STF therefore distinguishes the ambient product space from the subset of admissible tokens.

Let

$$\mathcal{M}_{\mathrm{STF}} \subset \mathcal{T}$$

be the *valid token manifold*. Formally, let

$$\mathcal{K} = \{\kappa_1, \ldots, \kappa_m\}$$

be a family of well-formedness predicates

$$\kappa_j : \mathcal{T} \to \{0, 1\}.$$

Then

$$\mathcal{M}_{\mathrm{STF}} = \{t \in \mathcal{T} \mid \kappa_j(t) = 1 \text{ for all } \kappa_j \in \mathcal{K}\}.$$

The manifold view captures the fact that the coordinates are coupled. A valid token cannot be formed by arbitrary independent choice of lexical span, syntax, semantics, and logical relation. Examples of admissibility constraints include:

- lexical spans must correspond to source-realizable regions;
- syntactic paths must be compatible with document structure;
- semantic content must be licensed by lexical and structural evidence;
- logical endpoints must refer to admissible semantic carriers;
- coherence must certify compatibility of the remaining coordinates.

The manifold is therefore the correct mathematical object for reasoning about semantic tokens. The full product space defines available dimensions; the manifold defines valid realizations.

8 Coupling Functions

The dimensions of STS are not independent. Their dependencies are expressed by coupling functions.

Semantic Coupling

Semantic interpretation is determined by lexical evidence, syntactic structure, and context:

$$\Sigma = f_\Sigma(L, S, C), \qquad f_\Sigma : \mathcal{L} \times \mathcal{S} \times \mathcal{C} \to \mathcal{S}_{\mathrm{em}}.$$

In the current implementation, f_Σ is approximated by deterministic normalization of lexical content, with context treated implicitly and syntax used primarily to delimit compilation units.

Logical Coupling

Logical structure is induced by semantics, context, and pragmatic framing:

$$\Gamma = f_\Gamma(\Sigma, C, P), \qquad f_\Gamma : \mathcal{S}_{\mathrm{em}} \times \mathcal{C} \times \mathcal{P} \to \mathcal{G}.$$

This function captures the fact that relations such as containment, ordering, support, reference, or contradiction are not purely lexical phenomena. In the current baseline, only a minimal deterministic fragment is realized.

Pragmatic Coupling

Pragmatic intent depends on context, meaning, and logical structure:

$$P = f_P(C, \Sigma, \Gamma), \qquad f_P : \mathcal{C} \times \mathcal{S}_{\text{em}} \times \mathcal{G} \to \mathcal{P}.$$

This formalizes the observation that the same propositional content can function as an assertion, question, warning, or instruction depending on discourse conditions and relational placement.

Coherence Coupling

Global coherence is a function of all prior coordinates:

$$\Omega = f_\Omega(L, S, \Sigma, \Gamma, C, P, \Delta),$$

with

$$f_\Omega : \mathcal{L} \times \mathcal{S} \times \mathcal{S}_{\text{em}} \times \mathcal{G} \times \mathcal{C} \times \mathcal{P} \times \mathcal{D} \to \mathcal{O}.$$

Operationally, f_Ω is approximated by validation and retention: schema validity determines ω_{schema}, invariants determine ω_{inv}, and retention induces ω_ρ.

These functions encode the central claim of STS: semantic tokens are coupled objects rather than independent feature tuples.

9 Operations

The manifold $\mathcal{M}_{\text{STF}}$ admits several natural operations.

Composition

Composition is a partial binary operation

$$\oplus : \mathcal{M}_{\text{STF}} \times \mathcal{M}_{\text{STF}} \rightharpoonup \mathcal{M}_{\text{STF}}.$$

For compatible tokens t_1 and t_2, the composition

$$t_3 = t_1 \oplus t_2$$

is defined only when lexical, structural, contextual, and coherence constraints permit a valid merged token. Informally, composition forms a larger semantic unit from smaller admissible units.

Equivalence

Semantic equivalence is defined by

$$t_1 \equiv_\Sigma t_2 \quad \Longleftrightarrow \quad \pi_\Sigma(t_1) =_{\mathcal{S}_{\text{em}}} \pi_\Sigma(t_2),$$

where $=_{\mathcal{S}_{\text{em}}}$ is a chosen semantic equivalence relation on $\mathcal{S}_{\text{em}}$.

Logical equivalence is defined by

$$t_1 \equiv_\Gamma t_2 \quad \Longleftrightarrow \quad \pi_\Gamma(t_1) \cong \pi_\Gamma(t_2),$$

where $\cong$ denotes equality of logical role up to irrelevant identifier renaming or equivalent typed adjacency structure.

Reduction

For a retained dimension set

$$r \subseteq \{L, S, \Sigma, \Gamma, C, P, \Delta, \Omega\},$$

define the reduction or projection operator

$$\pi_r(t) = (\pi_X(t))_{X \in r}.$$

Its codomain is the product of the retained coordinate spaces. Reduction forgets selected coordinates while preserving the retained view. It is the correct notation for dimension-wise projection and is intentionally distinct from the retention metric ρ.

Enrichment

Enrichment is a monotone endomorphism

$$\eta : \mathcal{M}_{\mathrm{STF}} \to \mathcal{M}_{\mathrm{STF}}$$

such that identity is preserved while information may increase in selected coordinates. A typical enrichment preserves lexical anchoring and structural position while refining semantics, logic, context, or coherence. Formally, one expects

$$\pi_L(\eta(t)) = \pi_L(t), \qquad \pi_S(\eta(t)) = \pi_S(t),$$

and

$$\pi_\Sigma(t) \preceq \pi_\Sigma(\eta(t)).$$

Analogous monotonicity conditions may be imposed on Γ, C, P, Δ, or Ω depending on the enrichment regime.

10 Semantic Conservation

Composition should preserve or improve semantic information in the order on $\mathcal{S}_{\mathrm{em}}$. Assume that local joins exist where composition is defined, and write $\sqcup$ for the semantic join or least admissible upper bound. Then the conservation principle is

$$\Sigma(t_1 \oplus t_2) \succeq \Sigma(t_1) \sqcup \Sigma(t_2).$$

This expresses monotonicity of semantic composition: a valid composition should not lose constituent meaning unless an explicit reduction is applied afterward.

The coherence bound is conservative rather than optimistic. With the componentwise order on $\mathcal{O}$, require

$$\Omega(t_1 \oplus t_2) \leq \min\bigl(\Omega(t_1), \Omega(t_2)\bigr),$$

where min is taken componentwise. The interpretation is that composition should not be assumed to improve coherence automatically. It can preserve or degrade coherence, but it should not exceed the weaker constituent without additional evidence.

These principles together distinguish enrichment from uncontrolled transformation: semantic content should be monotone under valid composition, while coherence remains bounded by validation constraints.

11 Mapping to Implementation

The formal model is intended to remain aligned with the current deterministic implementation. Table 1 summarizes the correspondence.

A few interpretive remarks are important.

First, `ZToken` materializes the operational slice of the theory rather than the full STS tuple. It explicitly realizes lexical, syntactic, and semantic information, and carries local logical references. Second, the global logical structure is split between artifact-level relations and the graph projection. Third, coherence is not

Formal object	Implementation surface	Role in the current system
L	`ZToken.lexical`	Source text, spans, normalized text
S	`ZToken.syntactic`	Node type, depth, path, parent reference
Σ	`ZToken.semantic`	Deterministic gloss fallback
Γ	`ZToken.logical`, `Artifact.relations`, `SirGraph.edges`	Local logical references and global typed relations
z	`ZToken`	Stable compiled unit
$\mathcal{A}$	`Artifact` / `.zmd`	Canonical typed artifact and serialization
$\Pi_{\Sigma,\Gamma}(\mathcal{A})$	`Artifact::as_sir_graph()`	Deterministic graph projection
ω_{schema}	schema validator	Artifact shape and required fields
ω_{inv}	invariants validator	Structural and semantic well-formedness
ω_ρ	retention baseline	Deterministic coherence proxy
ρ	`Artifact::retention_baseline()`	Dimension-wise preservation baseline

Table 1: Mapping from the STS formalization to the current STF-SIR implementation.

a standalone runtime field in the same sense as lexical or syntactic data; it is assembled operationally from validation and retention.

The mapping is therefore faithful but intentionally incomplete. STS generalizes the implementation without contradicting it.

12 Applications

The main practical value of the framework is that it provides a deterministic semantic substrate between raw documents and downstream computation. These applications emerge from the existence of a stable, addressable semantic substrate.

Structured retrieval and RAG. A retrieval system may index compiled units rather than raw chunks. This supports retrieval over typed spans, relations, and graph neighborhoods instead of only surface proximity. The result is a retrieval regime that is better aligned with semantic locality.

Agent reasoning. Agents can consume stable compiled structures rather than reparsing raw context on every step. This enables explicit reference to tokens, relations, and graph neighborhoods, and supports validation before action selection.

Semantic differencing. Because tokens, relations, and serialization are deterministic, changes between artifacts can be expressed at the level of semantic units and relation structure rather than only at the byte or line level.

Merge systems. Composition and coherence constraints suggest a path toward merge operators that reason about structural and semantic compatibility, not only textual overlap.

Validation pipelines. The explicit coherence coordinate and retention baseline make it possible to gate downstream processing on artifact validity and preservation thresholds.

These applications do not require the full future theory. They follow already from the deterministic baseline, provided that the intermediate representation remains stable and analyzable.

13 Limitations

The present framework has several deliberate limitations.

- The current semantic layer is simplified: Σ is centered on a deterministic gloss fallback rather than a full compositional semantic interpretation.
- Context C is largely implicit. The implementation captures ordering and containment but not rich discourse or external grounding.
- Pragmatics P is not explicitly implemented.
- Temporal state Δ is only weakly represented through versioning and provenance metadata.
- The current logical layer is minimal. In the baseline, relations are largely structural, and no inference is performed.
- Retention is an internal completeness metric, not a benchmark-grounded measure of semantic correctness.

The current system prioritizes determinism and structural fidelity over semantic completeness. These limitations are not accidental omissions; they are part of the decision to establish a stable reference baseline before enlarging the semantic scope.

14 Future Work

Several extensions follow naturally from the current formulation.

- **Explicit context.** The coordinate C can be extended to model discourse scope, antecedent structure, retrieval state, and inter-token contextual dependency.
- **Pragmatics.** The coordinate P can be made explicit through discourse-act typing, communicative force, and role-sensitive annotations.
- **Temporal evolution.** The coordinate Δ can support semantic versioning, artifact lineage, token-level provenance, and temporal validity intervals.
- **Advanced retention.** The metric layer can move beyond completeness checks toward corpus-grounded and task-grounded preservation analysis.
- **Semantic query engines.** The formalization supports query languages over tokens, relations, and graph projections, including semantic pattern matching and constrained composition.

A central design requirement for all such extensions is monotonicity with respect to existing artifacts. Future versions should enrich the representational space without invalidating the v1 baseline.

15 Related Work

STF-SIR sits at the intersection of tokenization, intermediate representations, graph models, and semantic evaluation.

Compared with subword tokenization methods such as byte-pair encoding and WordPiece, the framework changes the objective. Subword methods optimize sequence encodability and vocabulary efficiency; STF-SIR optimizes explicit semantic structure, validation, and reproducibility.

Compared with abstract syntax trees, STF-SIR shares the compiler-oriented intuition that structure should be made explicit. However, ASTs are primarily syntactic objects, whereas STF-SIR integrates syntax with semantic content, typed relations, and preservation metrics.

Compared with compiler intermediate representations such as Static Single Assignment (SSA) and LLVM IR, the framework shares several design commitments: explicit structure, analyzable transformation boundaries, stable identities, and reproducible compilation under fixed parameters. The difference is domain and semantic target. Compiler IRs are designed for executable programs and optimization pipelines, whereas STF-SIR targets semantically structured documents and AI-facing meaning preservation.

The framework is also related to graph-based intermediate representations, in which dependencies are made explicit as nodes and edges rather than left implicit in linear form. In that respect, SirGraph should be understood as a conservative graph IR over compiled semantic artifacts rather than as a general-purpose inferred knowledge graph.

Compared with knowledge graphs, SirGraph is intentionally conservative. It does not perform entity linking, inference, ontology completion, or graph augmentation. It is a deterministic projection of a compiled artifact rather than a global world model.

Compared with embedding-based representations, STF-SIR favors explicit symbolic structure over latent geometric similarity. Embeddings are powerful for approximate matching and generalization, but they do not, by themselves, provide typed invariants, stable identifiers, or deterministic serialization.

The contribution of the present framework is therefore not to supersede these paradigms, but to introduce a layer that connects them: a semantic intermediate representation that is deterministic like a compiler IR, structured like a graph view, and analyzable through explicit preservation metrics.

16 Conclusion

This paper has presented a complete formal account of STF-SIR and its extension to the Semantic Token Space. The operational baseline defines a deterministic compiler from source documents to semantic artifacts composed of ztokens, typed relations, and validation metadata. The graph layer, SirGraph, realizes the relation-bearing view of those artifacts without adding inference. The metric layer, ρ, provides a deterministic baseline for dimension-wise preservation analysis. The theoretical layer, STS, generalizes the operational model into a multidimensional semantic space with a valid token manifold, coupling functions, and algebraic operations.

The resulting picture is cohesive: STF is the compilation framework, SIR is the deterministic intermediate representation, SirGraph is its graph projection, and ρ is its preservation metric. STS provides the formal space in which these components can be understood as parts of a single mathematical and engineering program. This combination yields a stable baseline for future work on structured retrieval, agent grounding, semantic differencing, and richer semantic query systems.